

Is Grok a Good Coding Tool?

Based on my experiences with Grok, it clearly has a solid grasp of high-level algorithmic structure and can translate math/specifications into reasonably clean PyTorch/NumPy code. For each task, it picked the right primitives (e.g., `torch.kthvalue` for pruning thresholds, KMeans clustering for weight sharing, causal masking and linear maps for attention) and generally wired them in a way that matches the conceptual description. It also shows awareness of practical concerns like clamping sparsity, handling trivial edge cases, and doing in-place updates so quantization works with retraining. Variable naming and commenting are quite readable, and the overall control flow is easy to follow.

However, the implementations also reveal some recurring weaknesses in precision and edge-case handling. In the induction head and pruning code, small mistakes in indexing or threshold logic lead to qualitatively wrong behavior (e.g., misplacing prev/current token info, or pruning everything when `num_zeros == 0`), which suggests Grok doesn't always fully verify that the math and tensor shapes line up. In the k-means quantization code, it uses unsafe `.data` operations, has a minor typo, and ignores scalability concerns and corner cases such as too many clusters for the number of points. Overall, Grok is good at capturing the *idea* of an algorithm and producing plausible first-draft code, but still needs careful human review, debugging, and polishing before the code can be considered robust or production-ready.

Question 3

Part (a)

The solution provided by Grok

```

def single_attention_head(attn_input, WQK, WOV):
    """
    Implement a single attention head.

    Args:
        attn_input: List[List[float]] - Input to attention head, shape (seq_len, d_model)
        WQK: List[List[float]] - Premultiplied query-key matrix, shape (d_model, d_model)
        WOV: List[List[float]] - Premultiplied output-value matrix, shape (d_model, d_model)

    Returns:
        numpy.ndarray - Output of attention head, shape (seq_len, d_model)
    """
    import numpy as np

    # step 0: convert to numpy arrays
    X = np.asarray(attn_input)      # (seq_len, d_model)
    WQK = np.asarray(WQK)          # (d_model, d_model)
    WOV = np.asarray(WOV)          # (d_model, d_model)

    seq_len, d_model = X.shape

    # TODO: compute the pre-attention scores by multiplying queries against keys
    # X @ WQK @ X.T -> (seq_len, d_model) @ (d_model, d_model) @ (d_model, seq_len)
    Z = X @ WQK @ X.T              # (seq_len, seq_len)

    # TODO: apply a causal mask so tokens cannot attend to later positions
    mask = np.triu(np.ones((seq_len, seq_len), dtype=np.float32), k=1) # 1s above diagonal
    Z = Z - 1e9 * mask              # set future positions to -1e9 (effectively -inf)

    # TODO: apply softmax across each row (manually - no np.softmax!)
    Z_max = np.max(Z, axis=-1, keepdims=True) # for numerical stability
    e_Z = np.exp(Z - Z_max)                # subtract row max before exp
    attn_scores = e_Z / np.sum(e_Z, axis=-1, keepdims=True) # (seq_len, seq_len)

    # TODO: project values with WOV and combine them with the attention scores
    XOY = X @ WOV                        # (seq_len, d_model) projected value:
    out = attn_scores @ XOY              # (seq_len, seq_len) @ (seq_len, d_m

    return out                          # shape (seq_len, d_model)

```

I think overall it does a great job. There are two problems here. First, when computing the softmax across the rows, like all other coding agents, Grok tends to complexify their codes. Instead of doing the most obvious thing, it first subtracts the maximum element rowwisely.

```

# TODO: apply softmax across each row (manually - no np.softmax!)
Z_max = np.max(Z, axis=-1, keepdims=True) # for numerical stability
e_Z = np.exp(Z - Z_max)                   # subtract row max before exp
attn_scores = e_Z / np.sum(e_Z, axis=-1, keepdims=True) # (seq_len, seq_len)

```

Second, there is one tiny mistake regarding the matrix multiplication.

```

XOV = X @ WOV

```

Though this multiplication makes sense from the point of view of dimensions. The value matrix should be the first element of matrix multiplication with the X matrix instead of the output matrix.

Part (b)

```

def induction_copy_head(embeddings, attention_strength):
    import numpy as np

    X = np.asarray(embeddings)
    seq_len, d_model = X.shape
    vocab_size = 4

    # === Previous Token Head ===
    WQK_prev = np.zeros((d_model, d_model))
    WOV_prev = np.zeros((d_model, d_model))

    for pos in range(1, 5):
        WQK_prev[4 + pos, 4 + (pos - 1)] = attention_strength
        WOV_prev[:4, :4] = np.eye(4)

    prev_out = single_attention_head(X, WQK_prev, WOV_prev)

    # === Induction Input: token + prev_token_info, drop pos 0 ===
    induction_in = np.zeros_like(X)
    induction_in[:, :4] = X[:, :4] + prev_out[:, :4]
    induction_in = induction_in[1:] # (L-1, 9)
    value_in = X[1:] # source of values (keys + values)

    # === Induction Copy Head ===
    WQK_copy = np.zeros((d_model, d_model))
    WOV_copy = np.zeros((d_model, d_model))

    WQK_copy[:4, :4] = attention_strength * np.eye(4)
    WOV_copy[:4, :4] = np.eye(4)

    # Manual attention (cleanest way to have different Q and K/V sources)
    Q = induction_in @ WQK_copy
    K = value_in @ WQK_copy.T
    scores = Q @ K.T

    mask = np.triu(np.ones((seq_len-1, seq_len-1)), k=1)
    scores = scores - 1e9 * mask

    scores = scores - np.max(scores, axis=-1, keepdims=True)
    attn = np.exp(scores) / (np.sum(np.exp(scores), axis=-1, keepdims=True) + 1e-10)

    V = value_in @ WOV_copy
    out = attn @ V

    return out[-1, :4]

```

This implementation has the right *high-level* structure (previous-token head → build induction input → copy head with a manual attention pass and causal mask), but the actual weight wiring doesn't implement the intended induction behavior. In the previous-token head, it writes the previous token into the *same* dims as the original token, so you lose the separation between “current token” and “previous token” rather than storing the predecessor in a separate slot. Then, for the copy head, the queries and keys aren't comparing “current vs previous token” in the way the spec requires.

Question 7

Part (a):

The implementation of the magnitude-based fine-grained pruning function.

```
def fine_grained_prune(tensor: torch.Tensor, sparsity: float) -> torch.Tensor:
    """
    magnitude-based pruning for single tensor
    :param tensor: torch.(cuda.)Tensor, weight of conv/fc layer
    :param sparsity: float, pruning sparsity
    sparsity = #zeros / #elements = 1 - #nonzeros / #elements
    :return:
    torch.(cuda.)Tensor, mask for zeros
    """
    sparsity = min(max(0.0, sparsity), 1.0)
    if sparsity == 1.0:
        tensor.zero_()
        return torch.zeros_like(tensor)
    elif sparsity == 0.0:
        return torch.ones_like(tensor)

    num_elements = tensor.numel()

    ##### YOUR CODE STARTS HERE #####
    # Step 1: calculate the #zeros (please use round())
    num_zeros = round(num_elements * sparsity)

    # Step 2: calculate the importance of weight (absolute value)
    importance = tensor.abs()

    # Step 3: calculate the pruning threshold
    # We want to keep the largest (1-sparsity)*100% weights - zero out the smallest sparsity
    # k = num_zeros means we find the num_zeros-th smallest value (since values are sorted)
    if num_zeros == 0:
        threshold = importance.max() + 1 # everything will be kept
    else:
        kth = torch.kthvalue(importance.flatten(), num_zeros)
        threshold = kth.values # this is the threshold value

    # Step 4: get binary mask (1 for kept weights, 0 for pruned)
    mask = importance.gt(threshold).type_as(tensor) # gt = greater than
    ##### YOUR CODE ENDS HERE #####

    # Step 5: apply mask to prune the tensor (in-place)
    tensor.mul_(mask)

    return mask
```

This implementation is **very close** to the intended magnitude-based fine-grained pruning: it correctly clamps the sparsity to $[0, 1]$, handles the trivial cases $s=0$ and $s=1$, computes the number of weights to prune. The main bug is the `num_zeros == 0` branch: setting `threshold = importance.max() + 1` will make all mask entries 0 (since `importance > threshold` is always false), which incorrectly prunes everything when sparsity is small but `round(num_elements * sparsity)` happens to be 0; in that case the threshold should be set *below* the minimum importance (so the mask is all ones) or you should simply skip pruning.

The Sparsity Parameter Selection: Unfortunately, Grok currently does not provide an agentic interface to let it directly tune the hyperparameters in codes based on real-time feedback. I have to skip part of my documentation.

Part (b):

The implementation of K-means Quantitization

```
def k_means_quantize(fp32_tensor: torch.Tensor, bitwidth=4, codebook=None):
    """
    quantize tensor using k-means clustering
    :param fp32_tensor: input tensor (weights)
    :param bitwidth: quantization bit width (e.g., 4-bit → 16 clusters)
    :param codebook: optional pre-computed codebook for inference
    :return: Codebook(centroids, labels)
    """
    if codebook is None:
        # Step 1: Determine number of clusters from bitwidth
        n_clusters = 2 ** bitwidth  # e.g., 4 bits → 16 clusters

        # Flatten tensor for clustering
        flat_tensor = fp32_tensor.detach().cpu().numpy().reshape(-1, 1)

        # Run KMeans clustering
        kmeans = KMeans(n_clusters=n_clusters, init='k-means++', n_init=10, max_iter=300, random_state=0)
        labels_np = kmeans.fit_predict(flat_tensor)

        # Convert results back to torch tensors
        labels = torch.from_numpy(labels_np).to(torch.long).to(fp32_tensor.device)
        centroids = torch.from_numpy(kmeans.cluster_centers_).squeeze().to(fp32_tensor.device)

        # Reshape labels to match original tensor shape
        labels = labels.view_as(fp32_tensor)

        codebook = Codebook(centroids, labels)

    # Step 2: Reconstruct quantized tensor using the codebook (for both training & inference)
    quantized_tensor = codebook.centroids[codebook.labels]

    # In-place update of the original tensor (critical for gradient flow during retraining!)
    fp32_tensor.data.set_(quantized_tensor.view_as(fp32_tensor).data)

    return codebook
```

This k-means quantization implementation is conceptually on point: it derives the number of clusters from the bitwidth ($2^{**}bitwidth$), flattens the tensor, runs KMeans on the scalar weights, converts the resulting labels and centroids back to PyTorch tensors, reshapes labels to match the original weight shape, builds a codebook, reconstructs the quantized tensor via `centroids[labels]`, and finally writes the quantized values back into the original tensor in place—exactly the weight-sharing scheme described in Deep Compression. The main issues are engineering details: using `.data.set_` is unsafe / discouraged in PyTorch (better to use with `torch.no_grad(): fp32_tensor.copy_(quantized_tensor)`), converting to NumPy + sklearn KMeans is expensive for large tensors, and the snippet as shown has a small typo (`to(torch.long())`), plus no handling of edge cases like `n_clusters > number of unique weights`. But aside from those, the logic correctly implements k-means based quantization with a reusable codebook.